

DecisionTreeClassifier#

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

Description:

A decision tree classifier. Read more in the User Guide. The function to measure the quality of a split. Supported criteria are “gini” for the Gini impurity and “log_loss” and “entropy” both for the Shannon information gain, see Mathematical formulation. The strategy used to choose the split at each node. Supported strategies are “best” to choose the best split and “random” to choose the best random split.

Function Signature

```
class sklearn.tree.DecisionTreeClassifier(*,  
criterion='gini', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1,  
min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None,  
min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0,  
monotonic_cst=None)[source]#
```

Parameters:

criterion{“gini”, “entropy”, “log_loss”}, default=“gini”

```
splitter{"best", "random"}, default="best"
```

```
max_depthint, default=None
```

```
class sklearn.tree.DecisionTreeClassifier(*,  
criterion='gini', splitter='best', max_depth=None,  
min_samples_split=2, min_samples_leaf=1,  
min_weight_fraction_leaf=0.0, max_features=None,  
random_state=None, max_leaf_nodes=None,  
min_impurity_decrease=0.0, class_weight=None, ccp_alpha=0.0,  
monotonic_cst=None)[source]#
```

Parameters:

```
criterion{"gini", "entropy", "log_loss"}, default="gini"
```

```
splitter{"best", "random"}, default="best"
```

```
max_depthint, default=None
```

```
apply(X, check_input=True)[source]#
```

Parameters:

X{array-like, sparse matrix} of shape (n_samples,
n_features)

```
check_inputbool, default=True
```

Returns:

Parameters

ccp_alpha

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. See Minimal Cost-Complexity Pruning for details. See Post pruning decision trees with cost complexity pruning for an example of such pruning. Added in version 0.22.

monotonic_cstarray

Indicates the monotonicity constraint to enforce on each feature. 1: monotonic increase 0: no constraint -1: monotonic decrease If `monotonic_cst` is `None`, no constraints are applied. Monotonicity constraints are not supported for: multiclass classifications (i.e. when `n_classes > 2`), multioutput classifications (i.e. when `n_outputs_ > 1`), classifications trained on data with missing values. The constraints hold over the probability of the positive class. Read more in the User Guide. Added in version 1.4.

X{array}

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csc_matrix`.

X_leavesarray

For each datapoint `x` in `X`, return the index of the leaf `x` ends up in. Leaves are numbered within `[0; self.tree_.node_count)`, possibly with gaps in the numbering.

`X{array}`

The training input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csc_matrix`.

`yarray`

The target values (class labels) as integers or strings.

`sample_weightarray`

Sample weights. If `None`, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. Splits are also ignored if they would result in any single class carrying a negative weight in either child node.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`X{array}`

The training input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csc_matrix`.

yarray

The target values (class labels) as integers or strings.

sample_weightarray

Sample weights. If None, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. Splits are also ignored if they would result in any single class carrying a negative weight in either child node.

paramsdict

Parameter names mapped to their values.

X{array

The input samples. Internally, it will be converted to dtype=np.float32 and if a sparse matrix is provided to a sparse csr_matrix.

yarray

The predicted classes, or the predict values.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`X{array}`

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

`Xarray`

Test samples.

`yarray`

True labels for X.

`sample_weightarray`

Sample weights.

`**paramsdict`

Estimator parameters.

Code Examples

```
N_t / N * (impurity - N_t_R / N_t * right_impurity
           - N_t_L / N_t * left_impurity)
```

```
>>> from sklearn.datasets import load_iris
>>> from sklearn.model_selection import cross_val_score
>>> from sklearn.tree import DecisionTreeClassifier
>>> clf = DecisionTreeClassifier(random_state=0)
>>> iris = load_iris()
>>> cross_val_score(clf, iris.data, iris.target, cv=10)
...
...
array([ 1.        ,  0.93,  0.86,  0.93,  0.93,
        0.93,  0.93,  1.        ,  0.93,  1.        ])
```

Notes & Warnings

NOTE The search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

NOTE See also `DecisionTreeRegressor` A decision tree regressor.

Detailed Documentation

Documentation

A decision tree classifier.

Read more in the User Guide.

The function to measure the quality of a split. Supported criteria are “gini” for the Gini impurity and “log_loss” and “entropy” both for the Shannon information gain, see Mathematical formulation.

The strategy used to choose the split at each node. Supported strategies are “best” to choose the best split and “random” to choose the best random split.

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

The minimum number of samples required to split an internal node:

If int, then consider min_samples_split as the minimum number.

If float, then min_samples_split is a fraction and $\text{ceil}(\text{min_samples_split} * n_samples)$ are the minimum number of samples for each split.

Changed in version 0.18: Added float values for fractions.

The minimum number of samples required to be at a leaf node. A split point at any depth will only be considered if it leaves at least min_samples_leaf training samples in each of the left and right branches. This may have the effect of smoothing the model, especially in regression.

If int, then consider `min_samples_leaf` as the minimum number.

If float, then `min_samples_leaf` is a fraction and $\text{ceil}(\text{min_samples_leaf} * n_samples)$ are the minimum number of samples for each node.

Changed in version 0.18: Added float values for fractions.

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

The number of features to consider when looking for the best split:

If int, then consider `max_features` features at each split.

If float, then `max_features` is a fraction and $\text{max}(1, \text{int}(\text{max_features} * n_features_in_))$ features are considered at each split.

If "sqrt", then $\text{max_features} = \text{sqrt}(n_features)$.

If "log2", then $\text{max_features} = \text{log2}(n_features)$.

If None, then $\text{max_features} = n_features$.

The search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than `max_features` features.

Controls the randomness of the estimator. The features are always randomly permuted at each split, even if `splitter` is set to "best". When $\text{max_features} < n_features$, the algorithm will select `max_features` at random at each split before finding the best split among them. But the best found split may vary across different runs, even if $\text{max_features} = n_features$. That is the case, if the improvement of the criterion is identical for several splits and one split has to be selected at random. To obtain a

deterministic behaviour during fitting, `random_state` has to be fixed to an integer. See Glossary for details.

Grow a tree with `max_leaf_nodes` in best-first fashion. Best nodes are defined as relative reduction in impurity. If `None` then unlimited number of leaf nodes.

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following:

where N is the total number of samples, N_t is the number of samples at the current node, N_{t_L} is the number of samples in the left child, and N_{t_R} is the number of samples in the right child.

N , N_t , N_{t_R} and N_{t_L} all refer to the weighted sum, if `sample_weight` is passed.

Added in version 0.19.

Weights associated with classes in the form `{class_label: weight}`. If `None`, all classes are supposed to have weight one. For multi-output problems, a list of dicts can be provided in the same order as the columns of `y`.

Note that for multioutput (including multilabel) weights should be defined for each class of every column in its own dict. For example, for four-class multilabel classification weights should be `[{0: 1, 1: 1}, {0: 1, 1: 5}, {0: 1, 1: 1}, {0: 1, 1: 1}]` instead of `[{1:1}, {2:5}, {3:1}, {4:1}]`.

The “balanced” mode uses the values of `y` to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`

For multi-output, the weights of each column of y will be multiplied.

Note that these weights will be multiplied with `sample_weight` (passed through the `fit` method) if `sample_weight` is specified.

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than `ccp_alpha` will be chosen. By default, no pruning is performed. See Minimal Cost-Complexity Pruning for details. See Post pruning decision trees with cost complexity pruning for an example of such pruning.

Added in version 0.22.

1: monotonic increase

-1: monotonic decrease

If `monotonic_cst` is `None`, no constraints are applied.

multiclass classifications (i.e. when `n_classes > 2`),

multioutput classifications (i.e. when `n_outputs_ > 1`),

classifications trained on data with missing values.

The constraints hold over the probability of the positive class.

Read more in the User Guide.

Added in version 1.4.

The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).

Return the feature importances.

The inferred value of `max_features`.

The number of classes (for single output problems), or a list containing the number of classes for each output (for multi-output problems).

Number of features seen during fit.

Added in version 0.24.

Names of features seen during fit. Defined only when `X` has feature names that are all strings.

Added in version 1.0.

The number of outputs when fit is performed.

The underlying Tree object. Please refer to `help(sklearn.tree._tree.Tree)` for attributes of Tree object and Understanding the decision tree structure for basic usage of these attributes.

A decision tree regressor.

The default values for the parameters controlling the size of the trees (e.g. `max_depth`, `min_samples_leaf`, etc.) lead to fully grown and unpruned trees which can potentially be very large on some data sets. To reduce memory consumption, the complexity and size of the trees should be controlled by setting those parameter values.

The `predict` method operates using the `numpy.argmax` function on the outputs of `predict_proba`. This means that in case the highest predicted probabilities are tied, the classifier will predict the tied class with the lowest index in `classes_`.

https://en.wikipedia.org/wiki/Decision_tree_learning

L. Breiman, J. Friedman, R. Olshen, and C. Stone, “Classification and Regression Trees”, Wadsworth, Belmont, CA, 1984.

T. Hastie, R. Tibshirani and J. Friedman. “Elements of Statistical Learning”, Springer, 2009.

L. Breiman, and A. Cutler, “Random Forests”,
https://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm

Return the index of the leaf that each sample is predicted as.

Added in version 0.17.

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

Allow to bypass several input checking. Don't use this parameter unless you know what you're doing.

For each datapoint `x` in `X`, return the index of the leaf `x` ends up in. Leaves are numbered within `[0; self.tree_.node_count)`, possibly with gaps in the numbering.

Compute the pruning path during Minimal Cost-Complexity Pruning.

See Minimal Cost-Complexity Pruning for details on the pruning process.

The training input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csc_matrix`.

The target values (class labels) as integers or strings.

Sample weights. If None, then samples are equally weighted. Splits that would create child nodes with net zero or negative weight are ignored while searching for a split in each node. Splits are also ignored if they would result in any single class carrying a negative weight in either child node.

Dictionary-like object, with the following attributes.

Effective alphas of subtree during pruning.

Sum of the impurities of the subtree leaves for the corresponding alpha value in `ccp_alphas`.

Return the decision path in the tree.

Added in version 0.18.

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

Allow to bypass several input checking. Don't use this parameter unless you know what you're doing.

Return a node indicator CSR matrix where non zero elements indicates that the samples goes through the nodes.

Build a decision tree classifier from the training set (X, y).

The training input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csc_matrix`.

The target values (class labels) as integers or strings.

Sample weights. If None, then samples are equally weighted. Splits that would create

child nodes with net zero or negative weight are ignored while searching for a split in each node. Splits are also ignored if they would result in any single class carrying a negative weight in either child node.

Allow to bypass several input checking. Don't use this parameter unless you know what you're doing.

Return the depth of the decision tree.

The depth of a tree is the maximum distance between the root and any leaf.

The maximum depth of the tree.

Get metadata routing of this object.

Please check User Guide on how the routing mechanism works.

A MetadataRequest encapsulating routing information.

Return the number of leaves of the decision tree.

Get parameters for this estimator.

If True, will return the parameters for this estimator and contained subobjects that are estimators.

Parameter names mapped to their values.

Predict class or regression value for X.

For a classification model, the predicted class for each sample in X is returned. For a regression model, the predicted value based on X is returned.

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

Allow to bypass several input checking. Don't use this parameter unless you know what you're doing.

The predicted classes, or the predict values.

Predict class log-probabilities of the input samples `X`.

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

The class log-probabilities of the input samples. The order of the classes corresponds to that in the attribute `classes_`.

Predict class probabilities of the input samples `X`.

The predicted class probability is the fraction of samples of the same class in a leaf.

The input samples. Internally, it will be converted to `dtype=np.float32` and if a sparse matrix is provided to a sparse `csr_matrix`.

Allow to bypass several input checking. Don't use this parameter unless you know what you're doing.

The class probabilities of the input samples. The order of the classes corresponds to that in the attribute `classes_`.

Return accuracy on provided data and labels.

In multi-label classification, this is the subset accuracy which is a harsh metric since you require for each sample that each label set be correctly predicted.

Mean accuracy of `self.predict(X)` w.r.t. `y`.

Configure whether metadata should be requested to be passed to the fit method.

Note that this method is only relevant when this estimator is used as a sub-estimator within a meta-estimator and metadata routing is enabled with `enable_metadata_routing=True` (see `sklearn.set_config`). Please check the User Guide on how the routing mechanism works.

The options for each parameter are:

True: metadata is requested, and passed to fit if provided. The request is ignored if metadata is not provided.

False: metadata is not requested and the meta-estimator will not pass it to fit.

None: metadata is not requested, and the meta-estimator will raise an error if the user provides it.

str: metadata should be passed to the meta-estimator with this give